The Feedback Vertex Set Problem: Theory, Algorithms, and Applications

Project Checkpoint Presentation

Your Name

Your University

January 26, 2026

Outline

Problem Definition

Overview

- Define the Feedback Vertex Set (FVS) problem and formal statement.
- Provide visual examples and intuition (cycles vs. forests).
- Highlight key properties: NP-completeness and FPT results.

What is the Feedback Vertex Set Problem?

Definition (Feedback Vertex Set)

Given an undirected graph $G = (V, E)$ and an integer k, find a set $S \subseteq V$ with $|S| \leq k$ such that $G - S$ is **acyclic** (a forest).

Formal Problem:

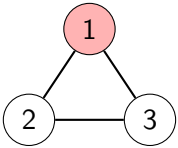
- **Input:** Graph $G = (V, E)$, integer k
- **Question:** $\exists S \subseteq V$, $|S| \leq k$ such that $G - S$ has no cycles?
- **Optimization:** Find minimum $|S|$

Intuition:

- Break all cycles by removing vertices
- Every cycle must contain ≥ 1 vertex from S
- Minimize disruption (minimize $|S|$)

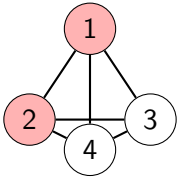
Visual Examples

Triangle (3-cycle)



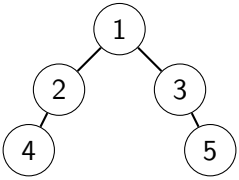
FVS= {1}, Size = 1

Complete Graph K_4



FVS= {1,2}, Size = 2

Tree



FVS= ∅, Size = 0

Key Property

A set S is a feedback vertex set if and only if removing S leaves no cycles.

Important Properties

1. **NP-Completeness:** FVS is NP-complete (Karp, 1972)
   - One of the original 21 NP-complete problems
2. **Parameterized Complexity:** Fixed-Parameter Tractable (FPT)
   - Solvable in $O(f(k) \cdot \text{poly}(n))$ time
   - Current best: $O(3.619^k \cdot n^4)$ [Kociumaka & Pilipczuk, 2010]
3. **Bounds:**
   - Lower: $|S| \geq |E| - |V| + c$ (where c = connected components)
   - Upper: $|S| \leq |V| - 1$
4. **Special Cases:**
   - Trees/Forests: $|S| = 0$
   - Planar graphs: Better approximation algorithms exist

Polynomial-Time Reductions

Overview

- Explain the role of reductions in proving hardness of FVS.
- Summarize key reductions (Vertex Cover, 3-SAT) and constructions.
- State implications for algorithmic transfer and complexity.

Reductions: Proving Hardness

Why Reductions Matter

- Prove FVS is NP-complete
- Show equivalence to other hard problems
- Transfer algorithmic techniques

Reductions TO FVS:

- Vertex Cover $\rightarrow \mathrm{FVS}$
- 3-SAT $\rightarrow \mathrm{FVS}$

Proves: FVS is NP-hard

Reductions FROM FVS:

- $\mathrm{FVS} \rightarrow$ Odd Cycle Transversal
- $\mathrm{FVS} \rightarrow$ Independent Set

Shows: Algorithmic connections

Reduction 1: Vertex Cover $\rightarrow$ FVS

Vertex Cover Problem

Given $G = (V, E)$, find minimum $S \subseteq V$ such that every edge has at least one endpoint in S.

Construction: Given Vertex Cover instance (G, k)

1. For each edge $e = (u, v) \in E$:
   - Create triangle with vertices $\{u, v, w_e\}$
   - Add edges: (u, v), (u, w_e), (v, w_e)
2. Set parameter: $k' = k$

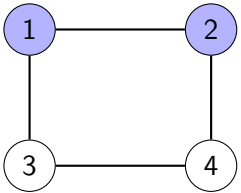
Theorem

G has vertex cover of size $\leq k \Leftrightarrow G'$ has FVS of size $\leq k$

Time: $O(|E|)$ — polynomial! ✓

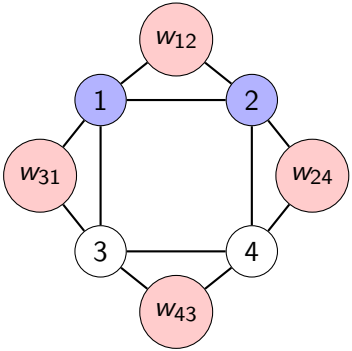
Vertex Cover $\rightarrow$ FVS: Example

Original Graph G:



Edges: (1,2), (2,4), (4,3), (3,1)
Vertex Cover: $\{1, 2\}$, size $= 2$

Transformed Graph G':



4 triangles created
FVS: $\{1, 2\}$ breaks all triangles, size $= 2$

Correctness

Reduction Summary

From	To	Time	Key Idea
Vertex Cover	FVS	$O(\|E\|)$	Replace edges with triangles
3-SAT	FVS	$O(n+m)$	Variable/clause gadgets
FVS	Odd Cycle Trans.	$O(\|V\|+\|E\|)$	Subdivide edges
FVS	Independent Set	$O(\|V\|+\|E\|)$	Complement problem

Implications

- FVS is as hard as other canonical NP-complete problems
- Techniques for one problem transfer to others
- Approximation hardness results transfer
- Establishes FVS as fundamental in complexity theory

Algorithm Survey

Overview

- Classify algorithms: Exact, Approximation, Randomized, Heuristic, Metaheuristic.
- Present representative methods and their time/quality trade-offs.
- Highlight algorithms selected for implementation and evaluation.

Five Major Categories of Solution Approaches

1. Exact Exponential

- Naive Brute Force
- DP on Subsets
- Iterative Compression
- Bounded Search Tree
- Current Best FPT
- Inclusion-Exclusion DP

2. Approximation

- 2-Approximation (VC-based)
- Log-Approximation
- LP Rounding
- Primal-Dual
- Local Ratio

4. Heuristic

- Greedy Vertex Selection
- Cycle-Based Greedy
- Maximum Degree Heuristic
- Core-Periphery Decomposition
- Reduction Rules + Greedy

5. Metaheuristic

- Genetic Algorithm
- Simulated Annealing
- Ant Colony Optimization
- Tabu Search
- Particle Swarm Optimization

Category 1: Exact Exponential Algorithms

Algorithm	Description	Complexity	Solution Quality
Naive Brute Force	Enumerate all 2^n subsets and check if each forms a valid FVS using cycle detection	$O(2^n \cdot n^3)$	Optimal
DP on Subsets	Dynamic programming to avoid recomputing subproblems; for each subset, determine if it's a valid FVS	$O(2^n \cdot n^2)$	Optimal
Iterative Compression	FPT algorithm that assumes solution of size $k+1$ and compresses to size k (Dehne et al., 2004)	$O(5^k \cdot kn^2)$	Optimal
Bounded Search Tree	Use branching rules to create search tree with depth $\leq k$; improved branching factor (Chen et al., 2006)	$O(4^k \cdot n^2)$	Optimal
Current Best FPT	State-of-art using crown decomposition, sunflower lemma, refined branching (Kociumaka & Pilipczuk, 2010)	$O(3.619^k \cdot n^4)$	Optimal
Inclusion-Exclusion DP	Uses inclusion-exclusion principle with DP for counting; fast for very small graphs ($n \leq 25$)	$O(2^n \cdot n)$	Optimal

Key Insight: FPT algorithms practical for $k \leq 50$; exponential in n feasible only for $n \leq 20$

Category 2: Approximation Algorithms

Algorithm	Description	Complexity	Solution Quality
2-Approximation (VC-based)	Reduce FVS to Vertex Cover; iteratively find cycles and remove both endpoints of any edge (Bafna et al., 1999)	$O(n^2)$	2-approx
Log-Approximation	For tournament graphs using hitting set techniques; specific to directed complete graphs	$O(n^2 \log n)$	$O(\log n)$-approx
LP Rounding	Formulate as ILP, solve LP relaxation, then round fractional solution (deterministic or randomized)	$O(n^{3.5})$	2-approx
Primal-Dual	Maintain primal and dual solutions simultaneously; combinatorial approach without LP solver	$O(n^2)$	2-approx
Local Ratio	Decompose weight function and apply local decisions; works well with weighted vertices	$O(n^2)$	2-approx

Open Question: Does $(2 - \varepsilon)$-approximation exist? Under UGC, 2 might be optimal.

Category 3: Randomized Algorithms

Algorithm	Description	Complexity	Solution Quality
Random Sampling	Randomly sample vertices in random order; if vertex in cycle, include it in FVS; Monte Carlo approach	$O(n^2)$ per trial	Expected $O(\log n)$-approx
Randomized Rounding	Solve LP relaxation, then randomly include vertex v with probability x_v^*; LP-based randomization	$O(n^3)$	Expected 2-approx
Color Coding	Randomly color vertices and use DP to find colorful solutions; for bounded treewidth graphs	$O(2^{O(k)} \cdot n)$	Optimal w.h.p.

Key Advantages

- Probabilistic guarantees with high probability (w.h.p.)
- Can be derandomized using method of conditional expectations
- Excellent for sparse graphs and bounded treewidth structures

Category 4: Heuristic Algorithms

Algorithm	Description	Complexity	Solution Quality
Greedy Vertex Selection	Repeatedly remove vertex with maximum degree; simple and fast; uses priority queue	$O(n^2 \log n)$	No guarantee
Cycle-Based Greedy	Find cycles using DFS/BFS and select vertex appearing in most cycles; iterative improvement	$O(n^3)$	No guarantee
Maximum Degree Heuristic	Like max degree but with tie-breaking: clustering coefficient, betweenness centrality	$O(n^2 \log n)$	No guarantee
Core-Periphery Decomposition	Compute k-core decomposition; focus on highly connected core vertices	$O(n + m)$	No guarantee
Reduction Rules + Greedy	Apply polynomial-time reduction rules first (degree 0,1,2 vertices, self-loops) then greedy	$O(n^2)$	No guarantee

Practical Value: Often reduce graph size by 50-90% before greedy phase; excellent performance on real-world networks

Category 5: Metaheuristic Algorithms

Algorithm	Description	Complexity	Solution Quality
Genetic Algorithm	Evolve population using selection, crossover, mutation; binary encoding or permutation-based	$O(g \cdot p \cdot n^2)$	No guarantee
Simulated Annealing	Probabilistic technique accepting worse solutions with decreasing probability; geometric cooling schedule	$O(i \cdot n^2)$	No guarantee
Ant Colony Optimization	Simulate ant foraging using pheromone trails; balance exploration vs exploitation	$O(\text{ants} \cdot i \cdot n^2)$	No guarantee
Tabu Search	Local search with memory (tabu list) to avoid cycling; aspiration criterion for best solutions	$O(i \cdot n^2)$	No guarantee
Particle Swarm Optimization	Swarm intelligence inspired by bird flocking; particles explore solution space collaboratively	$O(\text{particles} \cdot i \cdot n^2)$	No guarantee

Legend: g = generations, p = population size, i = iterations

Implementation Plan

Overview

- Describe selected algorithms to implement (Iterative Compression, Genetic Algorithm).
- Outline implementation details, complexity, and engineering considerations.
- State expected deliverables and milestones for development.

Selected Algorithms for Implementation

We will implement **two algorithms** from different categories:

Algorithm 1: Iterative Compression (Exact FPT)

Rationale:

- Demonstrates theoretical sophistication
- Guaranteed optimal solution
- Practical for moderate k values ($k \leq 30$)
- Showcases FPT technique

Complexity: $O(5^k \cdot kn^2)$

Algorithm 2: Genetic Algorithm (Metaheuristic)

Rationale:

- Practical for large instances

Why These Two Algorithms?

Complementary Strengths:

- Exact vs. Approximate
- Small vs. Large instances
- Theory vs. Practice
- Deterministic vs. Randomized

Learning Objectives:

- FPT algorithm design
- Evolutionary computation
- Parameter tuning
- Performance analysis

Experimental Comparison:

- Solution quality
- Runtime performance
- Scalability
- Robustness

Practical Relevance:

- IC: When optimality needed
- GA: When speed matters
- Covers industry use cases
- Publishable results

Alternative Choice

Could also implement: 2-Approximation + Simulated Annealing

Experimental Design

Overview

- Define datasets (synthetic + real-world) and instance sizes.
- Specify performance metrics: solution quality, runtime, robustness.
- Describe protocol: runs per instance, timeouts, reproducibility.

Dataset Design

Graph Type	Sizes	Instances	Purpose
Random (Erdős–Rényi)	$n \in \{10, 20, 50, 100, 200, 500, 1000\}$ $p \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$	350	General performance across densities
Scale-Free (BA)	$n \in \{10, 20, 50, 100, 200, 500, 1000\}$ $m \in \{2, 3, 5\}$	210	Realistic networks (power-law degree)
Grid	$\{3 \times 3, 5 \times 5, 10 \times 10,$ $20 \times 20, 30 \times 30\}$	25	Structured graphs with known properties
Small-World (WS)	$n \in \{20, 50, 100, 200, 500\}$ $k \in \{4, 6, 8\}, \beta \in \{0.1, 0.3, 0.5\}$	450	High clustering, short paths
Cycle-Heavy	4 base cycle counts 3 interconnection levels	120	Stress test (many cycles)
Trees	$n \in \{10, 20, 50, 100, 200\}$ Random, Balanced, Star	75	Baseline (optimal = 0)
Real-World	Karate, Dolphins, Email, Political blogs, Power grid	15-20	Validation on real networks

Total: $\sim$1,250 synthetic + 20 real-world instances

Performance Metrics

Solution Quality:

1. **FVS Size:** $|S|$ (smaller is better)
2. **Approximation Ratio:**

$$\text{Ratio} = \frac{|S_{\text{alg}}|}{|S_{\text{opt}}|}$$

3. **Relative Gap:**

$$\text{Gap} = \frac{|S_{\text{alg}}| - |S_{\text{best}}|}{|S_{\text{best}}|} \times 100\%$$

4. **Validity:** $G - S$ acyclic? (must be 100%)

Runtime Performance:

1. **Wall-Clock Time** (seconds)
2. **CPU Time** (process time)
3. **Iterations/Generations**
4. **Function Evaluations**

Robustness:

1. **Coefficient of Variation:**

$$CV = \frac{\sigma}{\mu} \times 100\%$$

2. **Success Rate** (solved within timeout)

Experimental Protocol

Implementation Environment

- **Language:** Python 3.x
- **Libraries:** NetworkX, NumPy, Matplotlib, Pandas
- **Hardware:** [Specify: CPU, RAM, OS]

Execution Details

- **Runs per instance:** 10 for randomized algorithms, 1 for deterministic
- **Timeout limits:**
  - Small ($n \leq 50$): 60 seconds
  - Medium ($50 < n \leq 200$): 300 seconds
  - Large ($n > 200$): 600 seconds
- **Random seeds:** Fixed for reproducibility
- **Data collection:** CSV format with all metrics

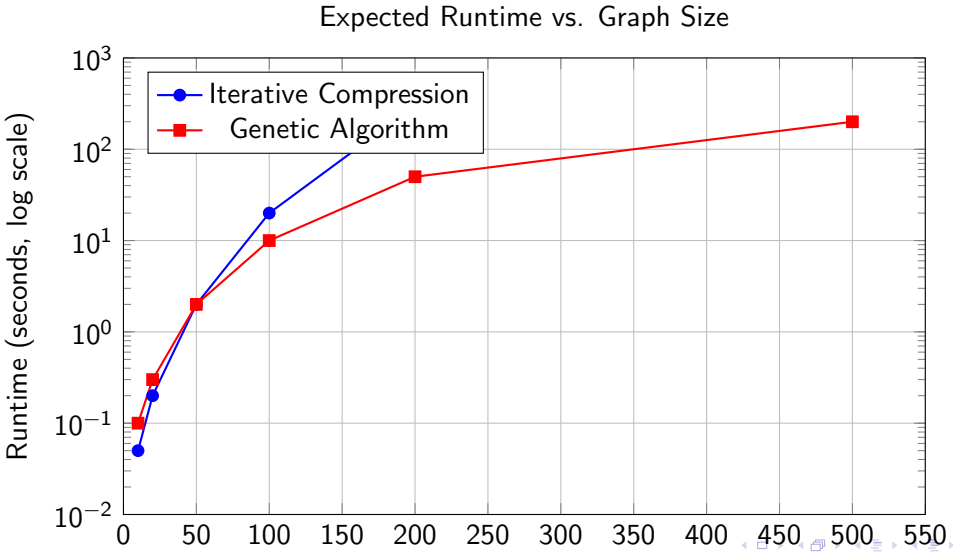
Key Experiments

1. **Correctness Verification**
   - Verify all solutions are valid FVS (100% required)
2. **Solution Quality Comparison**
   - Compare FVS sizes across algorithms
   - Box plots, statistical tests
3. **Runtime Performance**
   - Runtime vs. graph size (scalability)
   - Log-scale plots
4. **Quality-Runtime Trade-off**
   - Pareto frontier analysis
   - Best algorithm for given time budget
5. **Graph Type Sensitivity**
   - Performance heatmap across graph types
   - Identify algorithm strengths

Key Experiments (continued)

6. **Parameter Sensitivity (GA)**
   - Population size: $\{20, 50, 100, 200, 500\}$
   - Mutation rate: $\{0.01, 0.05, 0.1, 0.2\}$
   - Crossover rate: $\{0.5, 0.7, 0.8, 0.9\}$
7. **Convergence Behavior**
   - Track best solution over iterations/time
   - Convergence curves
8. **Optimality Gap (small instances)**
   - For $n \leq 20$: compute optimal with exact algorithm
   - Measure gap: (approx - optimal) / optimal
9. **Real-World Performance**
   - Validate on realistic networks
   - Compare to synthetic results

Expected Results Visualization



Applications

Overview

- Survey theoretical and practical applications of FVS.
- Provide domain examples: OS deadlocks, VLSI testing, biology, social networks, finance.
- Discuss impact and real-world relevance of solutions.

Application 1 : Deadlock Detection in Operating Systems

Problem

- Processes compete for shared resources
- Circular waiting leads to deadlock
- Goal: resolve deadlock with minimal impact

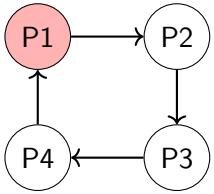
Graph Model

- Vertices represent processes
- Directed edges represent wait-for relations
- Cycles correspond to deadlocks

Role of FVS

- FVS models processes whose removal breaks all cycles

FVS $= \{P1\}$



Deadlock cycle in a wait-for graph

Application 2: VLSI Circuit Testing

Challenge

- Modern chips contain millions of logic gates
- Feedback loops make testing computationally hard
- Acyclic circuits are testable in linear time

FVS-Based Solution

- Model circuit as a graph
- Compute a small FVS to break all cycles
- Fix FVS gates and test remaining acyclic circuit
- Test FVS gates separately

Example

- 32-bit ALU circuit
- 5,000 gates total
- 200 feedback paths
- FVS size: 50 gates

Fixing 50 gates $\Rightarrow$ 4,950 acyclic gates

Benefits

- Reduced test time
- Lower power consumption
- Smaller hardware overhead

Application 3: Gene Regulatory Networks

Biological Motivation

- Genes regulate each other via feedback loops
- Cycles control cell behavior and disease states

Graph Model

- Vertices: genes
- Edges: regulatory interactions
- Cycles: feedback regulation

Role of FVS

- Small set of FVS genes controls all feedback : Interpreted as **master regulators**

Cancer Study Example

- Breast cancer regulatory network
- 500 genes analyzed
- FVS: 12 key genes
- 10 known and 2 novel drug targets

Applications

- Drug target discovery
- Gene knockout analysis
- Personalized medicine

Social and Financial Networks

Social Network Analysis

- Feedback loops amplify influence and information spread
- FVS nodes correspond to key influencers or superspreaders
- Targeting FVS improves marketing efficiency
- Disrupting FVS reduces misinformation propagation

Financial Systemic Risk

- Financial institutions form highly interconnected networks
- Cycles represent circular dependencies and systemic risk
- FVS identifies systemically important institutions
- Used for regulation, monitoring, and crisis prevention

Theoretical Applications

- **Complexity Theory**
  - Used in NP-completeness reductions
  - Benchmark problem for cycle-breaking hardness
- **Algorithm Design**
  - Testbed for FPT(Fixed Parameter Tractable) techniques
  - Motivates iterative compression and branching methods
- **Approximation Theory**
  - Used to study approximation limits
  - Reference problem for constant-factor approximations
- **Graph Structure Analysis**
  - Tool for analyzing treewidth and feedback structure
  - Applied in kernelization and graph decompositions

Potential Research Directions

Overview

- Highlight open theoretical problems (approximation gap, faster FPT, kernels).
- Point out practical directions: ML integration, dynamic and distributed algorithms.